

RUBY

Map : `map` transforms every element into something else and returns a new collection.

Big Picture

You now know the four most important Enumerable methods:

Method	Think of it as	Returns
<code>each</code>	Visit every element	Original collection
<code>map</code>	Transform every element	New collection
<code>select</code>	Keep matching elements	Filtered collection
<code>reduce</code>	Combine everything	Single value

You've Now Learned the Core Enumerable Methods

These are the methods you'll use constantly:

Method	Purpose	Returns
<code>each</code>	Visit every element	Original collection
<code>map</code>	Transform elements	New collection
<code>select</code>	Filter elements	New filtered collection
<code>reduce</code>	Combine elements	One final value
<code>group_by</code>	Organize into groups	Hash

Summary

WORLD 1 — &&

False dominates.

Right side isn't executed if left side is false.

WORLD 2 - ||

Means

If left is falsey, check right.

Real life

"Use WiFi OR Mobile Data."

Truth Dominates

Opposite of &&

|| returns first truthy value
or last falsey value

WORLD 3 – and

None

```
a = true and false
```

Ruby sees

None

```
(a = true) and false
```

```
a = true
```

WORLD 4 – or

None

```
user or return
```

Equivalent to

None

```
return unless user
```

WORLD 5 and 6 – & , |

These are bitwise operator

Ex. 5 | 3
#=>7

WORLD 8 — Block Passing

This confuses almost everyone.

```
None ▼   
def greet  
  yield  
end  
  
block = proc { puts "Hi" }  
  
greet(&block)
```

Here

&

does NOT mean AND.

It converts Proc into a block.

Opposite direction

```
None  
def greet(&block)  
  block.call  
end
```

Now block becomes Proc.

WORLD 7 — Arrays

Now same symbols change meaning.

Intersection

None

`a = [1,2,3]`

`b = [2,3,4]`

`a & b`

Output

None

`[2,3]`

Common elements.

Union

None

`a | b`

Output

None

`[1,2,3,4]`

Duplicates removed.

Edge case

None

[1,1,2] & [1]

Output

None

[1]

Duplicate removed.

Another

None

[1,1] | [1]

Output

None

[1]

Order preserved

None

[3,2,1] | [2,4]

Output

None

[3,2,1,4]

Note

Use `&&` and `||` for boolean expressions and assignments.

Use `and` and `or` only for control flow, if at all.

Many Rails developers avoid `and/or` entirely because `&&` and `||` are less error-prone.

Public Private and Protected

Comparison Table

Feature	Public	Protected	Private
Can be called from outside?	✓ Yes	✗ No	✗ No
Can be called inside same object?	✓ Yes	✓ Yes	✓ Yes
Can another object of the same class call it?	✓ Yes	✓ Yes	✗ No
Can use <code>self.method?</code>	✓ Yes	✓ Yes	✗ No

Can call without receiver?

✓ Yes

✓ Yes

✓ Yes

Rule to Remember

- **Public** → Anyone can call it.
- **Private** → Only the object itself can call it **without an explicit receiver**.
- **Protected** → Objects of the **same class (or subclasses)** can call it on each other, but code outside the class hierarchy cannot.

90% Developers ka Rule

Access	Kaun call kar sakta hai?
Public	Sab
Private	Sirf class ke andar bina <code>self</code> ke
Protected	Class ke andar aur same class ke doosre objects

Ruby ka rule hai

Private methods cannot receive messages from an explicit receiver.

Ye

None

```
check_balance
```

receiver nahi hai.

Ye

None

```
self.check_balance
```

Now let's understand **exactly why**.

Step 1

None

```
a.compare(b)
```

Memory:

None

```
a —————▶ Rahul Object  
                balance = 100
```

```
b —————▶ Amit Object  
                balance = 200
```

Ruby enters

None

```
def compare(other)
```

Now

None
`self`

points to

None
`Rahul Object`

and

None
`other`

points to

None
`Amit Object`

Step 2

Ruby reaches

None
`other.balance`

What is `other`?

None
`Amit Object`

So Ruby sees

None

```
Amit Object.balance()
```

Notice the **dot (.)**.

Whenever Ruby sees

None

```
something.method
```

it means

"Send a message to an object."

Here the object is **Amit**.

Step 3

Ruby checks:

Is `balance` private?

Yes.

Ruby's rule:

A private method cannot be called with an explicit receiver.

The explicit receiver here is

None

```
other
```

So Ruby immediately raises

None

```
private method 'balance' called
```

Hash

- **slice** → "Mujhe sirf ye pieces (keys) chahiye."
- **except** → "Mujhe sab chahiye, bas ye keys nahi."
- **compact** → "Jinki value **nil** hai, unko saaf kar do."

Methods You Should Know

Method	Purpose
<code>[]</code>	Read a value
<code>[]=</code>	Add or update a key
<code>fetch</code>	Read a value and optionally raise or provide a default
<code>delete</code>	Remove a key
<code>keys</code>	Return all keys
<code>values</code>	Return all values
<code>each</code>	Iterate over key-value pairs

<code>each_key</code>	Iterate over keys
<code>each_value</code>	Iterate over values
<code>merge</code>	Combine hashes
<code>invert</code>	Swap keys and values
<code>key? / has_key?</code>	Check if a key exists
<code>value? / has_value?</code>	Check if a value exists
<code>dig</code>	Safely access nested values
<code>transform_keys</code>	Change keys
<code>transform_values</code>	Change values
<code>select</code>	Keep matching pairs
<code>reject</code>	Remove matching pairs
<code>compact</code>	Remove pairs with <code>nil</code> values

<code>slice</code>	Return only specified keys
<code>except</code>	Return all except specified keys
<code>clear</code>	Remove all pairs
<code>replace</code>	Replace the entire hash

Arrays

Summary

Method	Purpose
<code>flatten</code>	Remove nesting
<code>compact</code>	Remove <code>nil</code>
<code>uniq</code>	Remove duplicates

<code>sort</code>	Sort values
<code>reverse</code>	Reverse order
<code>join</code>	Array → String
<code>zip</code>	Combine arrays
<code>product</code>	Cartesian product
<code>transpose</code>	Swap rows/columns
<code>combination</code>	Choose without order
<code>permutation</code>	Choose with order
<code>sample</code>	Random element
<code>shuffle</code>	Random order
<code>fill</code>	Replace values
<code>rotate</code>	Rotate elements

Time Complexity

Method	Complexity
--------	------------

<code>flatten</code>	$O(n)$
----------------------	--------

<code>compact</code>	$O(n)$
----------------------	--------

<code>uniq</code>	$O(n)$ average
-------------------	-------------------

<code>sort</code>	$O(n \log n)$
-------------------	---------------

<code>reverse</code>	$O(n)$
----------------------	--------

<code>join</code>	$O(n)$
-------------------	--------

<code>zip</code>	$O(n)$
------------------	--------

<code>product</code>	$O(n \times m)$
----------------------	-----------------

`sample` $O(1)$

`shuffle` $O(n)$
`e`

Message Passing(Imp)

In Ruby, you don't "call a function" on an object.

You **send a message** to an object.

None

`car.start`

Means:

None

`Object`

↓

```
"Please execute start."
```

Instance variable

Instance variables are **not accessible from outside the object**.

Instead, create a method.

```
None
```

```
class Car

  def initialize(color)

    @color = color

  end

  def color

    @color
```

```
end

end

car = Car.new("Red")

puts car.color
```

Class Getter n Setter

Summary

Feature	Purpose
<code>initialize</code>	Runs automatically when <code>.new</code> is called
<code>@variable</code>	Stores object-specific data

Getter	Read an instance variable
Setter	Change an instance variable
<code>attr_reader</code>	Generates getters
<code>attr_writer</code>	Generates setters
<code>Attr_access</code> or	Generates both

Famous Ruby Quote

"If it walks like a duck and quacks like a duck, treat it like a duck."

Meaning:

If an object behaves correctly,
its class doesn't matter.

Note

No method overloading in ruby

Last method replaces the previous ones

Interview Question

Q: Does Ruby require inheritance for polymorphism?

A: No.

Ruby primarily relies on **duck typing**. If an object responds to the required method, Ruby can use it regardless of its class or inheritance hierarchy. Inheritance is one way to achieve polymorphism, but it's not required.

Encapsulation

Encapsulation = Bundling data and methods together while hiding internal implementation details from the outside world.

HASH

JSON always gives String keys

None

```
require "json"
```

```
json = '{"name":"John"}'
```

```
h = JSON.parse(json)
```

```
h["name"]    # works
```

```
h[:name]     # nil
```

Private Methods

Imagine ATM software.

The customer should call:

None

```
withdraw()
```

The customer should **not** call:

None

```
verify_pin()
```

```
update_database()
```

```
send_sms()
```

```
record_transaction()
```

These are internal steps.

So make them private.

Example

None

```
class ATM
```

```
  def withdraw
```

```
    verify_pin
```

```
    update_balance
```

```
    puts "Cash Dispensed"
```

```
  end
```

```
  private
```

```
  def verify_pin
```

```
    puts "PIN Verified"
```

```
  end
```

```
def update_balance  
  puts "Balance Updated"  
end  
  
end
```

Outside:

```
None  
atm = ATM.new  
  
atm.withdraw
```

Protected Method

Protected methods are accessible:

- Inside the class
- By other objects of the same class
- By subclasses

• Summary

PUBLIC

Everyone can access

PROTECTED

Same class + subclasses

PRIVATE

Only inside same class
(without explicit receiver)

Can we create an object of Kernel?

No.

None

Kernel.new

Raises an error because modules cannot be instantiated.

Proc created inside method, called outside method

This is a very important edge case.

None

```
def create_proc  
  Proc.new do  
    return  
  end  
end  
  
x = create_proc  
x.call
```

This causes a `LocalJumpError`.